

项目申请书

项目名称: 恢复 openRuyi 基础组件的上游测试与构建根适配

项目编号: 2659a0007

项目主导师: misaka00251

申请人: 程祥伦

日期: 2026.08.04

邮箱: 2760018909@qq.com

1. 项目背景

1. 项目基本需求

1. 四个候选包的测试基线审计
2. 恢复 genimage 与 lua-json 上游测试
3. 恢复 LMDB / LibTomCrypt, 并记录另一包根因
4. 构建日志和测试跳过说明

2. 项目相关仓库与申请人基础

2. 技术方法及可行性

1. Linux 与 RISC-V 系统构建相关
2. 编程、脚本与构建调试相关
3. 容器、自动化与运维工具相关
4. 开源协作与项目匹配

3. 项目实施细节梳理

1. genimage / lua-json / LMDB / LibTomCrypt
2. 日志、跳过与量化验收

4. 规划

1. 第一阶段 (第 1-4 周)
2. 第二阶段 (第 5-8 周)
3. 期望

1. 项目背景

1. 项目基本需求:

项目仓库地址: <https://github.com/openRuyi-Project/openRuyi>

1. 当前问题与项目价值:

openRuyi 的部分基础软件包虽然能够完成 RPM 构建, 但仍保留空的 %check、只编译测试程序却不执行, 或沿用已经过时的测试豁免。这样只能证明源码可编译, 不能证明上游行为在当前构建根、当前依赖和目标架构下仍然正确。

本项目将把测试恢复为 RPM 的真实质量门: 上游测试必须在干净构建根中执行, 非豁免失败必须以非零退出码终止构建; 任何跳过都要说明前置条件、证据、影响和解除办法。全过程不依赖外网、特权容器、loop/mount 等宿主特权, 也不依赖宿主系统服务。

2. 项目基本需求:

- 完成 genimage、lua-json、LMDB、LibTomCrypt 四个候选包的测试基线审计, 固定源码、spec、工具链、依赖和可复现命令。
- 恢复 genimage 的 Automake 上游测试, 补齐 fakeroot 等实际测试依赖, 并保证失败会使 RPM 构建失败。
- 恢复 lua-json 的回归测试与 lunit 测试, 处理目标 Lua ABI、LUA_PATH/LUA_CPATH 和测试框架依赖。
- 优先恢复 LMDB 的 make test; 若实测表明 LibTomCrypt 更适合完整恢复, 经导师确认后交换两者角色。
- 对未进入完整恢复的另一核心包提供最小复现、根因链、影响范围、修复候选和维护建议。
- 提供构建日志索引、测试统计和逐项跳过清单, 不允许空 %check、宽泛的 || true 或静默跳过。

2. 技术方法及可行性

1. Linux 与 RISC-V 系统构建相关

本人具有较扎实的 Linux 使用与系统构建基础，能够在命令行环境中完成软件安装、文件与权限管理、进程和网络排查，并能结合日志定位常见环境问题。我已取得 RHCE 认证（2023），这些经验可用于搭建和检查 openRuyi 软件包的隔离构建环境。

- Linux 基础：熟悉常用命令、Shell 环境、文件系统、用户权限、软件包和服务管理，可分析构建日志与测试退出状态。
- RISC-V 实践：完成过从交叉编译工具链、Linux 内核、根文件系统到初始化系统的构建，并在模拟环境中启动和调试。
- 与项目的关系：能够区分宿主工具与目标架构产物，理解交叉编译、构建根和目标程序不可直接在宿主运行等问题。

2. 编程、脚本与构建调试相关

本人熟悉 Python，了解 C/C++，能够阅读并修改常见脚本、Makefile 和基础 C 语言测试代码。项目中可使用 Python 或 Shell 辅助整理环境指纹、执行命令、收集退出码和归档日志，并通过编译与链接信息定位测试入口、依赖或共享库路径问题。

- Python：可编写批量检查、日志整理、结果统计和可复现信息采集脚本，减少重复人工操作。
- C/C++ 与构建文件：能够阅读测试程序、Makefile、配置宏和编译参数，分析常见编译、链接与运行错误。
- Shell 与调试：能够组合环境变量、命令行工具和退出码完成测试复现，并保存可供维护者复核的命令记录。
- Git：能够使用分支、提交、diff、rebase 等基本流程，使补丁按依赖、测试入口和文档等主题拆分。

3. 容器、自动化与运维工具相关

本人熟悉 Docker、Docker Compose、Kubernetes 与 Ansible，并取得 CKS 认证（2024）。这些工具的使用经历使我对隔离环境、依赖声明、权限边界和自动化执行有基本认识，可以帮助识别测试对开发机残留状态、特权容器或宿主服务的隐式依赖。

- 容器：能够搭建相对干净的验证环境，检查环境变量、挂载、用户权限和宿主依赖。
- 自动化：能够使用 Ansible 或脚本固化重复步骤，便于在不同环境中复现同一测试过程。

- Kubernetes 与安全：了解容器运行时权限和安全边界，有助于判断测试是否确实需要特权能力。
- 工程记录：能够将 Coding、Testing、Docs 与 Debug 分开管理，并保留完整日志和问题处理记录。

4. 开源协作与项目匹配

本人 GitHub 账号为 corestudy，已向 openRuyi 软件包仓库提交 PR #351、#207、#179、#163，并向 homepage 仓库提交 PR #32，熟悉从 Fork、创建分支、提交修改到持续维护 Pull Request 的基本协作流程。

- 社区经验：能够根据仓库规范组织提交、回应评审意见，并在后续提交中修正问题。
- 项目匹配：已有 openRuyi 和 RISC-V 实践，可较快进入软件包 spec、构建日志和上游测试的实际工作。
- 学习计划：将在现有基础上进一步掌握 RPM spec、Automake、Lua 模块路径、ELF 动态加载以及 LMDB mmap/锁语义。

可行性说明：上述 Linux、RISC-V、编程脚本、容器自动化与开源协作经验，与本项目所需的构建根搭建、测试复现、日志分析和社区提交工作直接相关。

3. 项目实施细节梳理：

1. genimage：恢复 Automake 上游测试

genimage 上游 v19 的 Makefile.am 以 fakeroot 作为测试日志编译器，驱动 genimage.test、ext.test、filesystem.test、flash.test、hdimage.test、misc.test 等套件。当前历史 TODO 指向 fakeroot 缺口，因此先补齐可证明的 BuildRequires，再根据各脚本真实调用核验 e2fsprogs、util-linux 等工具。

1. 实施步骤：

- 固定上游版本与 spec，列出 TESTS、夹具、外部命令和每项能力要求。

- 补充 fakeroot 及实测需要的工具；构建产物优先加入 PATH，显式设置 srcdir/buildidir 和唯一 TMPDIR。
- 以源码树只读、不同工作目录和干净构建根三种场景执行测试，确认没有隐式路径依赖。
- 接入 %check 后注入一个确定失败，验证 RPM 中止；撤销后连续 3 次运行结果一致。

```
%check
make -j1 check
```

genimage 验收：六组上游套件逐项有 PASS/FAIL/SKIP 结果；非豁免失败阻断 RPM；skip 仅限确实无法满足的单项。

2. lua-json：恢复目标 Lua ABI 的回归测试

lua-json 上游 make check 由 regressionTest 与 lunit 测试组成。除 Lua 与 LPeg 外，回归测试需要 LuaFileSystem (lfs)，lunit 测试需要仓内 lua-lunitx 提供的 lua-lunit/lunit。实施时必须以 openRuyi 实际 Lua ABI 为准，不能用宿主全局 Lua 模块替代构建依赖。

1. 路径与依赖矩阵：

- 解释器：显式调用 spec 确定的 Lua，并记录 _VERSION。
- LUA_PATH：只指向源码/构建树中的 ?.lua 与 ?/init.lua，记录 json 模块实际来源。
- LUA_CPATH：仅在需要 C 扩展时指向目标 ABI 路径，禁止加载用户目录或其他 ABI 的 .so。
- 框架：先验证 make check-regression，再验证 lunit --interpreter 的仓内兼容性，最后执行完整 make check。
- 行为：保留错误输入、数字、Unicode、空值与嵌套结构测试，并记录测试/断言总数。

```
%check
export LC_ALL=C.UTF-8 TZ=UTC
export LUA_PATH="$PWD/?.lua;$PWD/?/init.lua;;"
make check
```

lua-json 验收：目标 Lua ABI 下 regression 与 lunit 真正执行；模块来源、套件统计和退出码可核对；确定失败可中止 RPM。

3. LMDB：优先恢复共享库路径下的实际测试

上游 libraries/liblmbd 的 make test 会重建 testdb、执行 mtest，再使用 mdb_stat 检查数据库。openRuyi 本地补丁把测试程序改为链接构建树中的 liblmbd.so，因此首要问题是确认运行时加载本次构建库，而不是宿主系统中的旧版本。

- 保存编译与链接命令，用 readelf/ldd 或 loader 调试信息核对 NEEDED 和实际解析对象。
- 在唯一、可写、非网络文件系统目录中设置 LD_LIBRARY_PATH=\$PWD，执行 make test。
- 核对 testdb 清理/重建、mtest 退出码和 mdb_stat 输出；记录 mmap、锁与文件系统环境。
- 设置错误库路径复现负向结果，再恢复正确路径，证明根因与修复有效。

```
%check
pushd libraries/liblmbd
LD_LIBRARY_PATH="$PWD${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}" make test
popd
```

LMDB 验收：mtest 与 mdb_stat 确实加载本次构建的 liblmbd.so；连续 3 次通过；失败原样传播到 RPM。

4. LibTomCrypt：根因、复现与维护建议

LibTomCrypt 的 make test 只负责生成测试程序，真正的算法自测需要继续运行 ./test，并从源码根读取 tests/test.key。项目将记录 makefile、CFLAGS、配置宏、libtommath/EXTRALIBS 和实际链接对象，避免把“测试程序编译成功”误写成“测试通过”。

- 在源码根执行 test -x ./test && ./test，保存退出码和完整输出。
- 核对测试所用静态/共享 libtomcrypt 与数学后端是否等同 RPM 交付配置。
- 若未作为第三个完整恢复包，仍交付最小复现、根因链、影响和可审阅补丁候选。

4. 规划：

项目周期为 2 个月（8 周）。总体工作量按 Coding 约 40%、Testing 约 30%、Docs 约 15%、Debug/返工约 15% 分配；每周以可复现证据和可评审产出为完成标准。

1. 项目研发第一阶段（第 1 周 - 第 4 周）：

- W1: 固定四包 commit/spec、工具链与依赖；建立环境指纹、日志索引和 skip 模板。
- W2: 完成上游/独立构建目录/RPM 构建根三层基线；复现第一失败点；确认 LMDB/LibTomCrypt 角色。
- W3: 补齐 genimage 测试依赖和路径适配；执行六组套件、负向失败和只读源码树验证。
- W4: 完成 genimage 连续 3 次回归；开始 lua-json ABI、LUA_PATH/LUA_CPATH 与框架兼容验证。

2. 项目研发第二阶段（第 5 周 - 第 8 周）：

- W5: 完成 lua-json regression 与 lunit 接入；验证目标 ABI、错误输入、Unicode 和模块来源。
- W6: 完成 LMDB 共享库路径适配、mtest/mdb_stat、错误库路径负向测试与 3 次回归。
- W7: 完成 LibTomCrypt 最小复现、链接/配置证据、根因与维护建议；对三个恢复包交叉回归。
- W8: 在全新构建根最终复验；整理补丁、完整日志、skip 清单、复现指南和项目总结。

3. 阶段验收与沟通：

- 每周至少一次向导师同步证据、阻塞和下一周目标；新增依赖或改变第三包选择时提前确认。
- W2 完成四包审计，W4 完成 genimage，W5 完成 lua-json，W6 完成第三包，W8 完成全部交付。
- 上游 PR 合并不作为唯一完成条件；本地补丁质量、干净根验证和维护者可复现记录为项目验收依据。

4. 时间余量：

各周任务内保留机动时间，并在 W4、W7-W8 集中安排 Debug、跨架构复验和评审返工，合计约占总周期 15%。若出现环境不可用等外部阻塞，优先保证四包基线、三个真实测试恢复和另一包根因记录不缩水。

5. 期望：

希望借助本项目深入参与 openRuyi 软件包维护，把历史空 %check 和模糊测试豁免转化为可执行、可失败、可复现的质量门。项目完成后，至少三个基础组件能够在不依赖外网、特权容器或宿主服务的前提下真实运行上游测试，剩余候选包也具备可操作的根因证据和维护建议。

我希望在导师与社区维护者的指导下，进一步掌握 RPM spec、Automake、Lua 模块系统、ELF 动态链接、RISC-V 构建环境和上游测试维护方法，并把形成的日志与跳过模板复用于 openRuyi 其他基础软件包。